



OBJECT ORIENTED ANALYSIS AND DESIGN

Ch - 1

A study material for the students of GLS University

Introduction to OOAD

- OOAD is based on the concepts of object-oriented programming (OOP) and is an organized and systematic approach to designing and developing software systems.
- Focuses on real-world modeling using classes and objects.
- Divided into two phases:
 - Object-Oriented Analysis (OOA)
 - Object-Oriented Design (OOD)

OOAD Phases

OOA (What):

- Finding and describing objects or concepts in problem domain
- Investigation of problem and requirements
- Requirement Analysis
- Object based analysis

OOD (How): Create class structures, define relationships, and apply design principles.

Real-World Example: Online Shopping

Imagine you're building Amazon-like software where customers can:

- Browse products
- Add them to cart
- Place an order
- Make payment

STEP 1: OBJECT-ORIENTED ANALYSIS (The 'What' Phase)

FUNCTIONAL REQUIREMENTS

-  Browse Products
-  Search Products
-  Order Payment
-  Payment

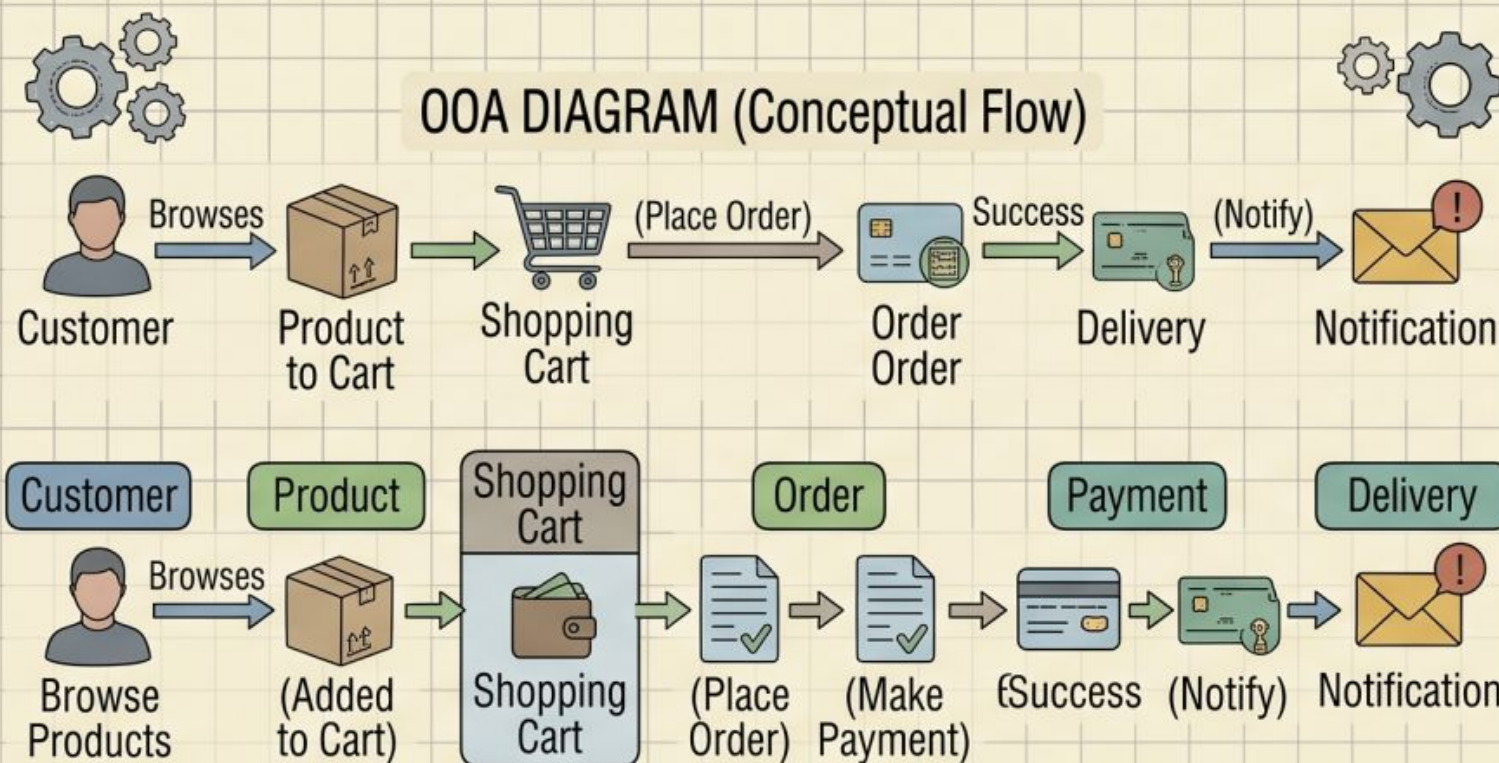
OOA Answers:

- Who are the users?
- What should they?
- What are the system requirements manifest present?
- What objects are present?

IDENTIFYING ACTORS

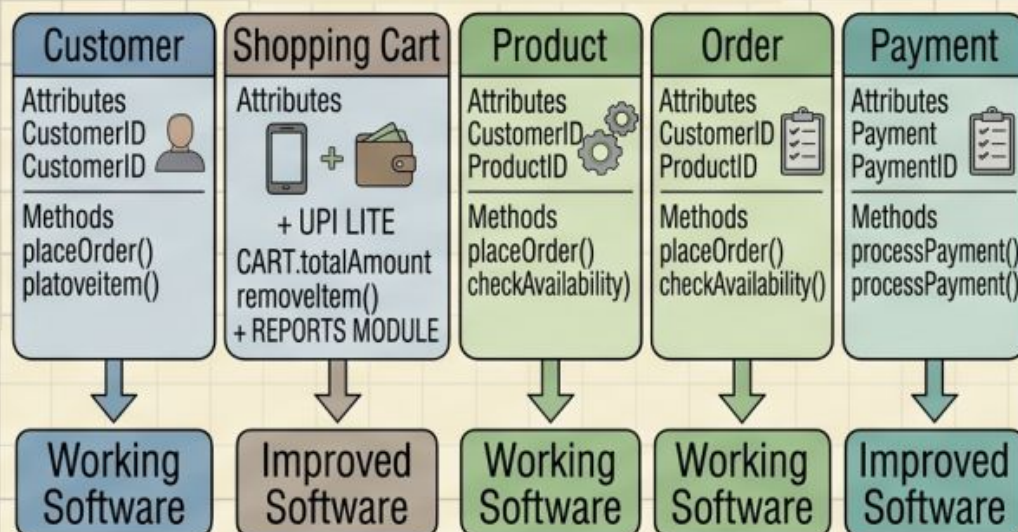
- | | |
|--|--|
|  CUSTOMER
Browse & Order |  ADMIN
Manage Products |
|  GATEWAY
Responsibilities |  Process Payments
Deliver |
|  PARTNER
Deliver |  SERVICE
Send Updates |

OOA DIAGRAM (Conceptual Flow)

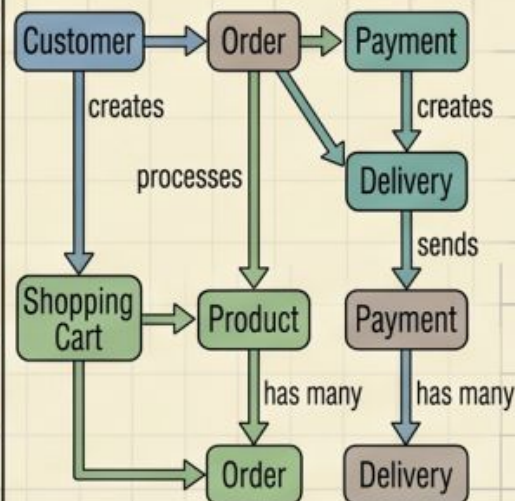


STEP 2: OBJECT-ORIENTED DESIGN (The 'How' Phase)

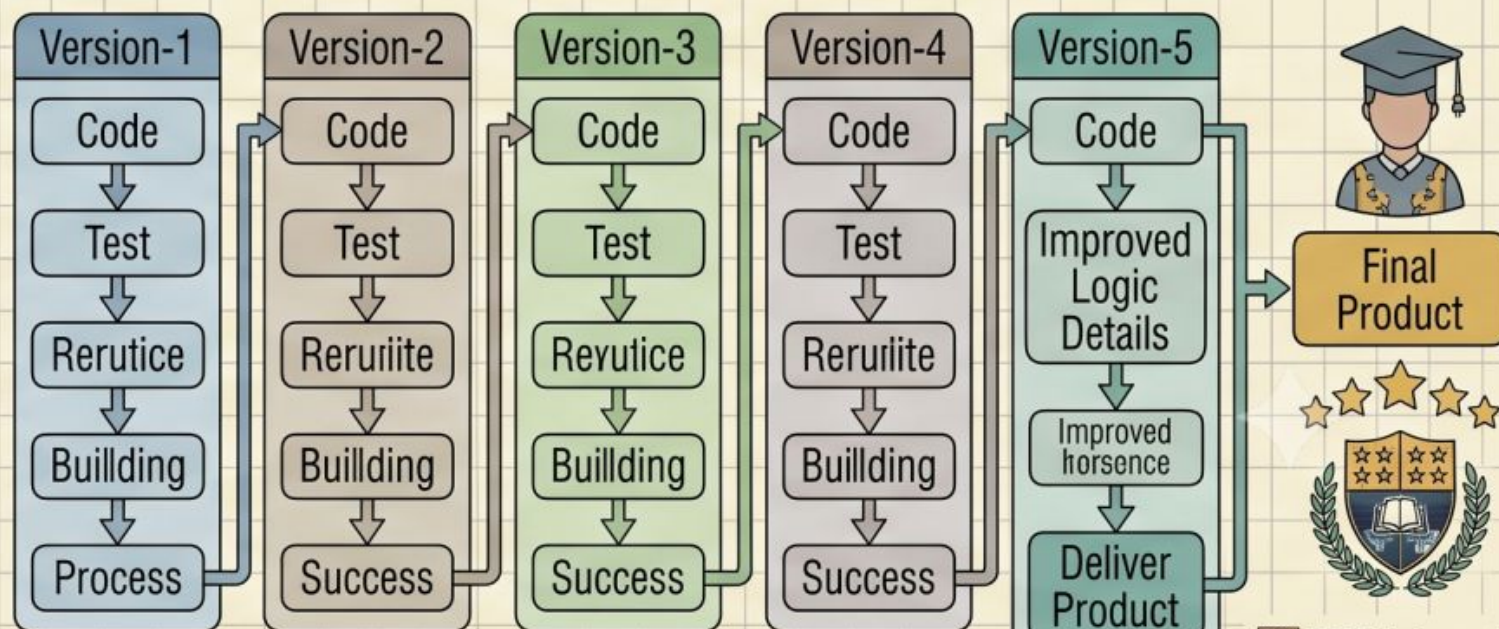
DESIGNING THE CLASSES



RELATIONSHIP IN OOD



SYSTEM WORKFLOW (Workflow & Pipeline)



Benefits of OOAD

- Reusability: Use of common classes in other projects
- Maintainability: Easy to change features (e.g., add reviews)
- Scalability: System grows without breaking
- Real-world mapping: Easier to design and understand

MCQs

1. What is the full form of OOAD?

- A. Object-Oriented Analysis and Design
- B. Object-Oriented Architecture and Development
- C. Object Organization and Development
- D. Object-Oriented Application Development

Ans: A. Object-Oriented Analysis and Design

2. Which concept binds data and methods together?

- A. Abstraction
- B. Inheritance
- C. Encapsulation
- D. Aggregation

Ans: C. Encapsulation

MCQs

3. What is the goal of Object-Oriented Analysis (OOA)?

- A. To implement classes and objects
- B. To create test cases
- C. To understand and model user requirements
- D. To optimize performance

Ans: C. To understand and model user requirements

4. What is the main goal of Object-Oriented Design (OOD)?

- A. Write documentation
- B. Convert analysis model into design model
- C. Compile the code
- D. Identify hardware resources

Ans: B. Convert analysis model into design model

MCQs

5. Which phase focuses on identifying "what" the system should do, not "how" to do it?

- A. OOD
- B. Coding
- C. OOA
- D. Testing

Ans: C. OOA

6. Which phase transforms requirements into a solution design using classes and objects?

- A. OOA
- B. OOD
- C. Coding
- D. Deployment

Ans: B. OOD

UML (Unified Modeling Language)

Content

1. What is UML?
2. The birth of UML
3. MDA – the future of UML
4. Objects and UML
5. UML Structure
6. UML building blocks
7. UML common mechanism
8. Architecture

What is a model?

- A model is a simplified or abstract representation of a real-world system. It helps us understand, analyse, communicate, and test different aspects of a system before it is actually built.
- Instead of dealing with all the complexities of a system, a model focuses only on the important details required for a particular purpose.

Types of Models—

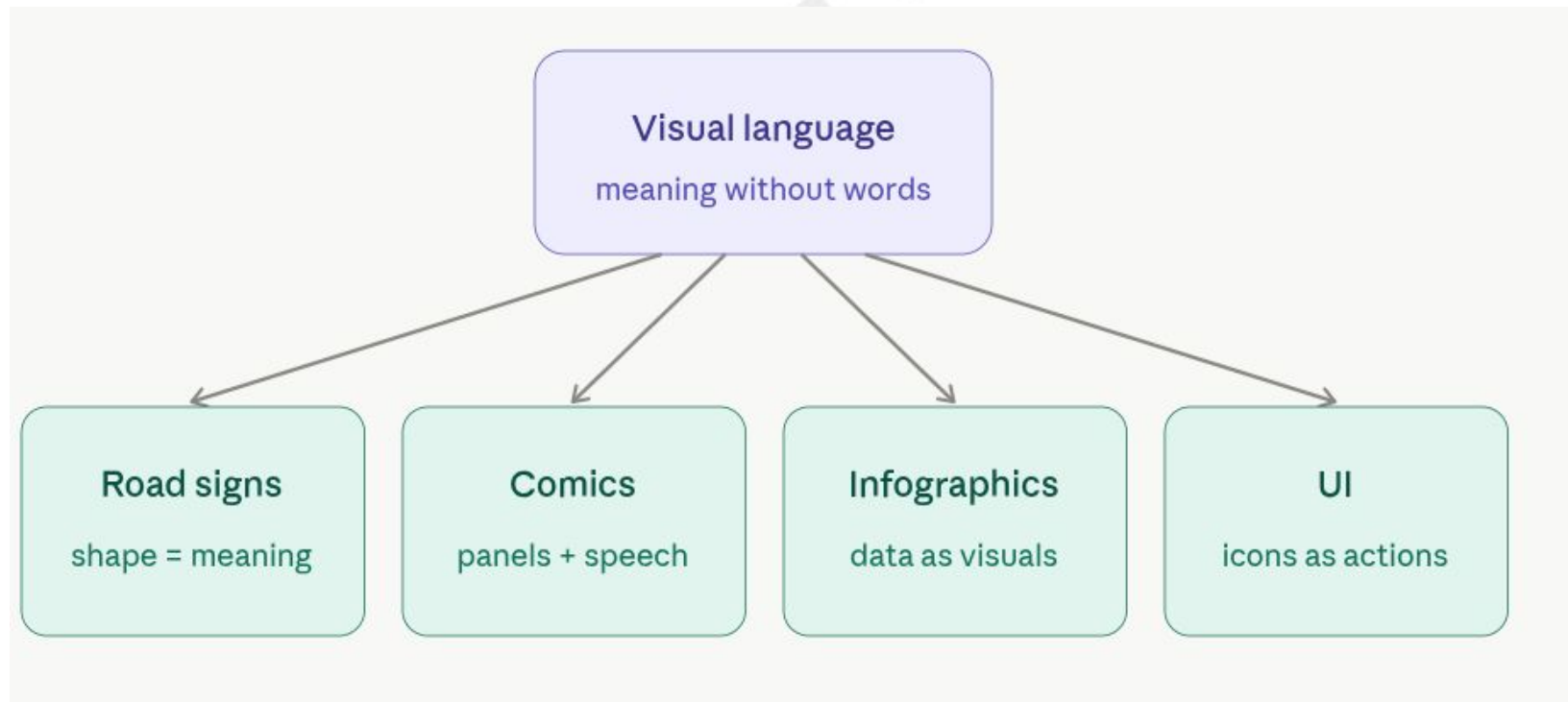
1. **Computational Models** – represents how a system behaves or performs computations
Examples: Flowcharts, Algorithms, Machine Learning models
2. **Analytical Models** - Analyze and evaluate the behaviour, performance, and efficiency of a system.
Examples: Performance analysis, Time complexity analysis, Network analysis
3. **Descriptive Models** - Describe the structure and behaviour of a system.
Examples: UML diagrams, Use Case Diagrams, Class Diagrams, Activity Diagrams

What is UML

- UML stands for Unified Modeling Language.
- UML is a standard visual language/ Graphical language used to:
 - Model (visualize) the structure and behavior of a software system
 - Document system design before actual coding
 - Help teams communicate design ideas clearly
- It is based on standards controlled **by Object Management Group (OMG)**.
- UML is like a blueprint or map for software. Just like architects use building plans before construction, software engineers use UML diagrams before writing code.

Visual Language

Visual language refers to a system of communication that uses visual elements—such as images, symbols, colors, layout, and design—to convey meaning, ideas, and emotions without (or with minimal) use of words.



Why Visual Language Matters

- universal understanding
- Faster Processing
- Emotionally Engaging
- Better Retention

Why UML was needed

- In the 1980s and early 1990s:
- Software systems were becoming more complex.
- Many object-oriented programming languages were emerging (like C++, Smalltalk).
- Multiple modeling methods were developed by different experts:

Booch Method (Grady Booch)

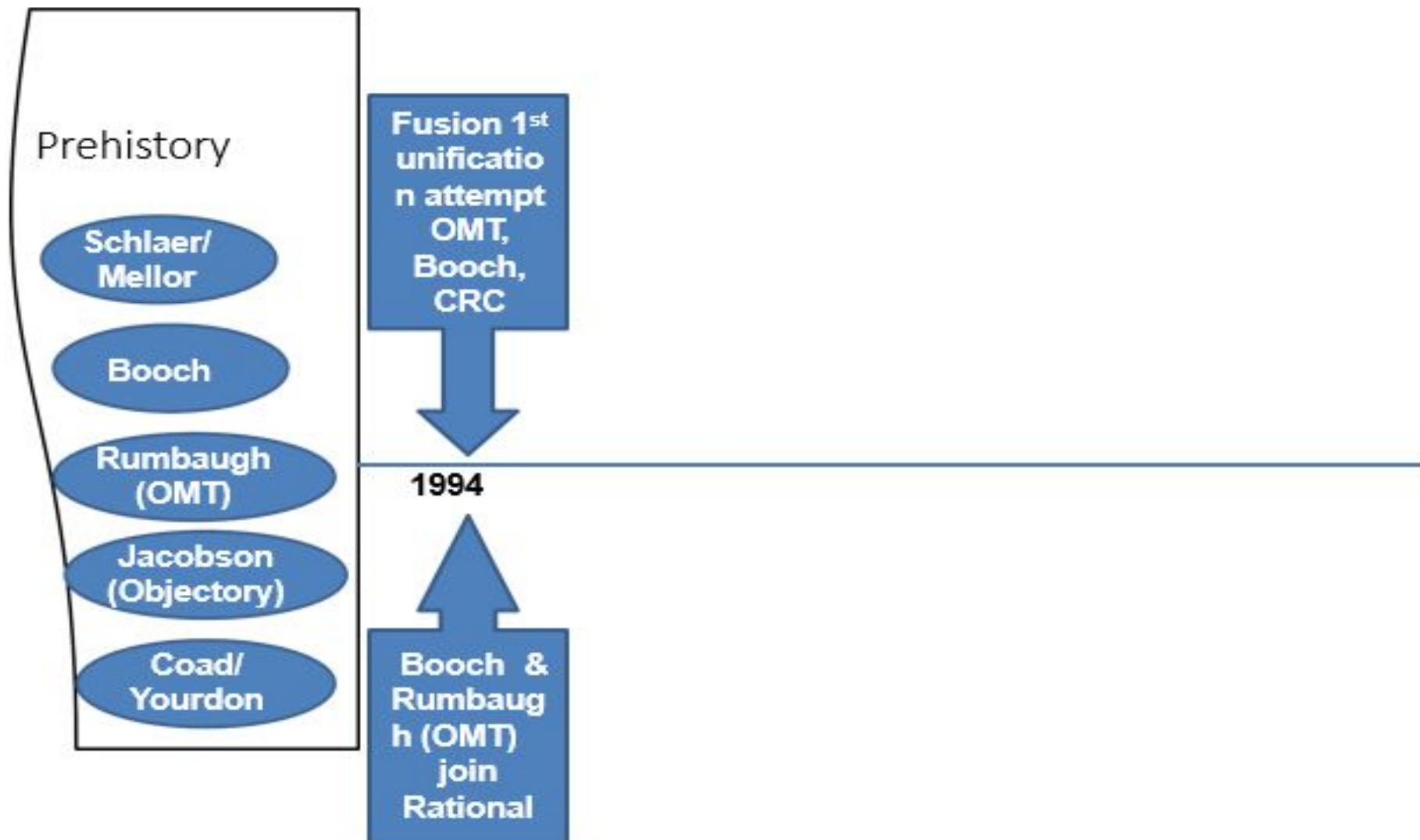
OMT – Object Modeling Technique (James Rumbaugh)

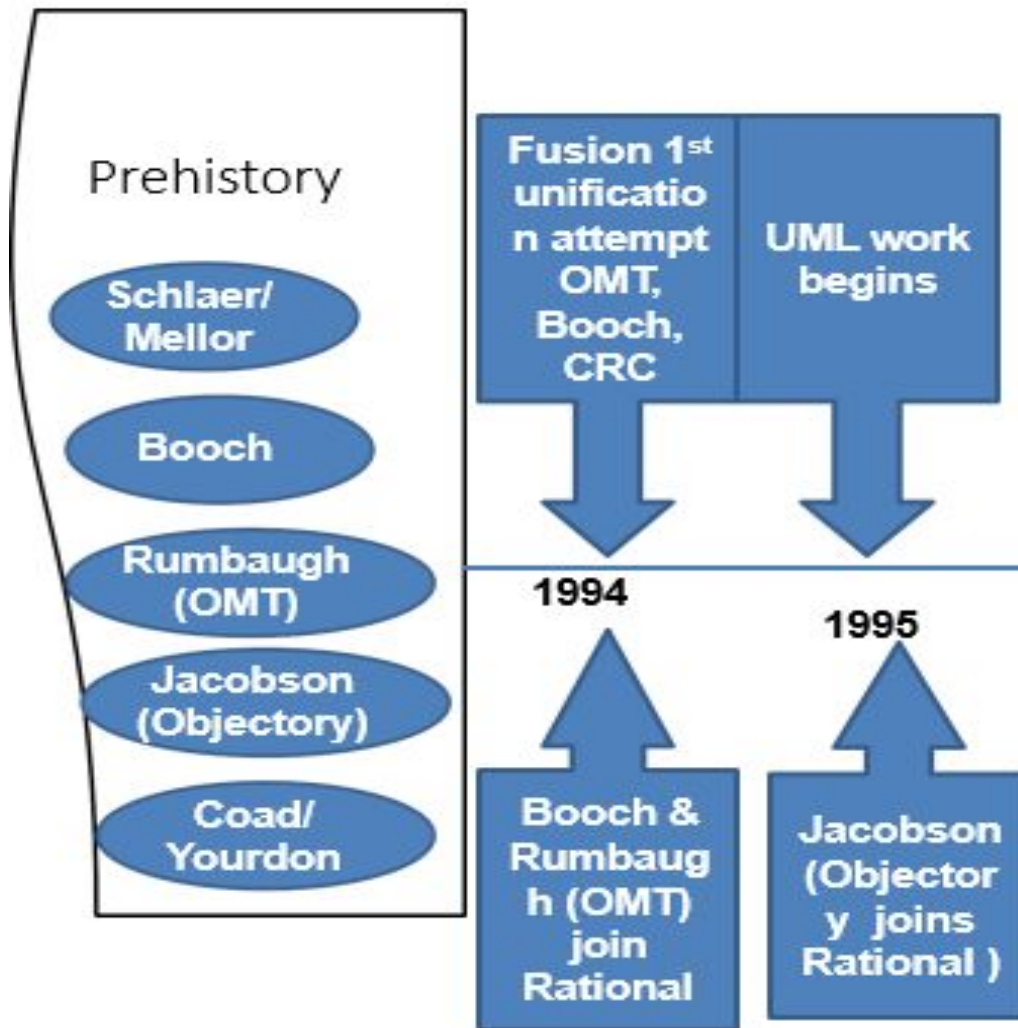
OOSE – Object-Oriented Software Engineering (Ivar Jacobson)

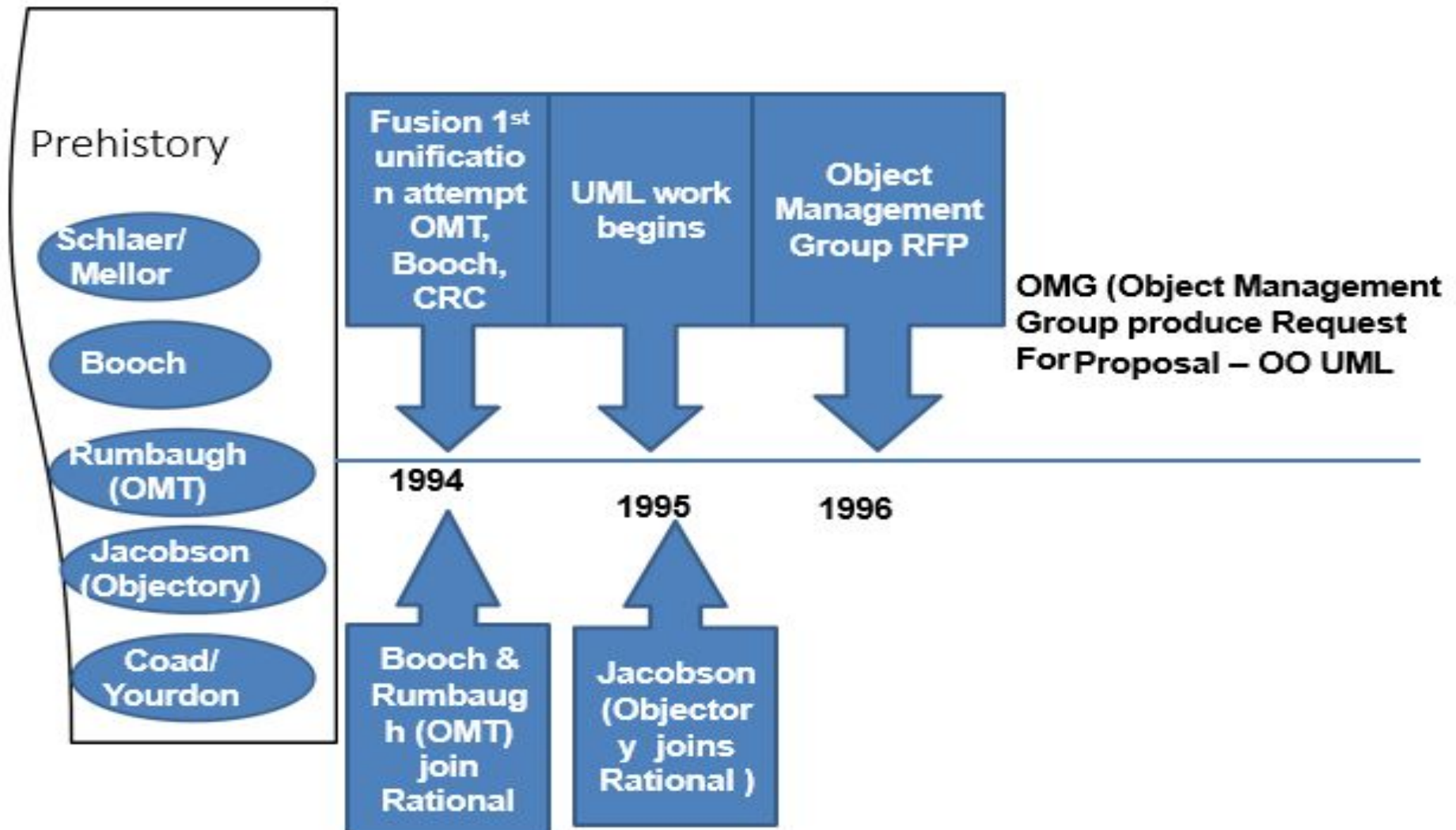
These methods all had different notations, tools, and styles. This led to confusion and inconsistency in design processes.

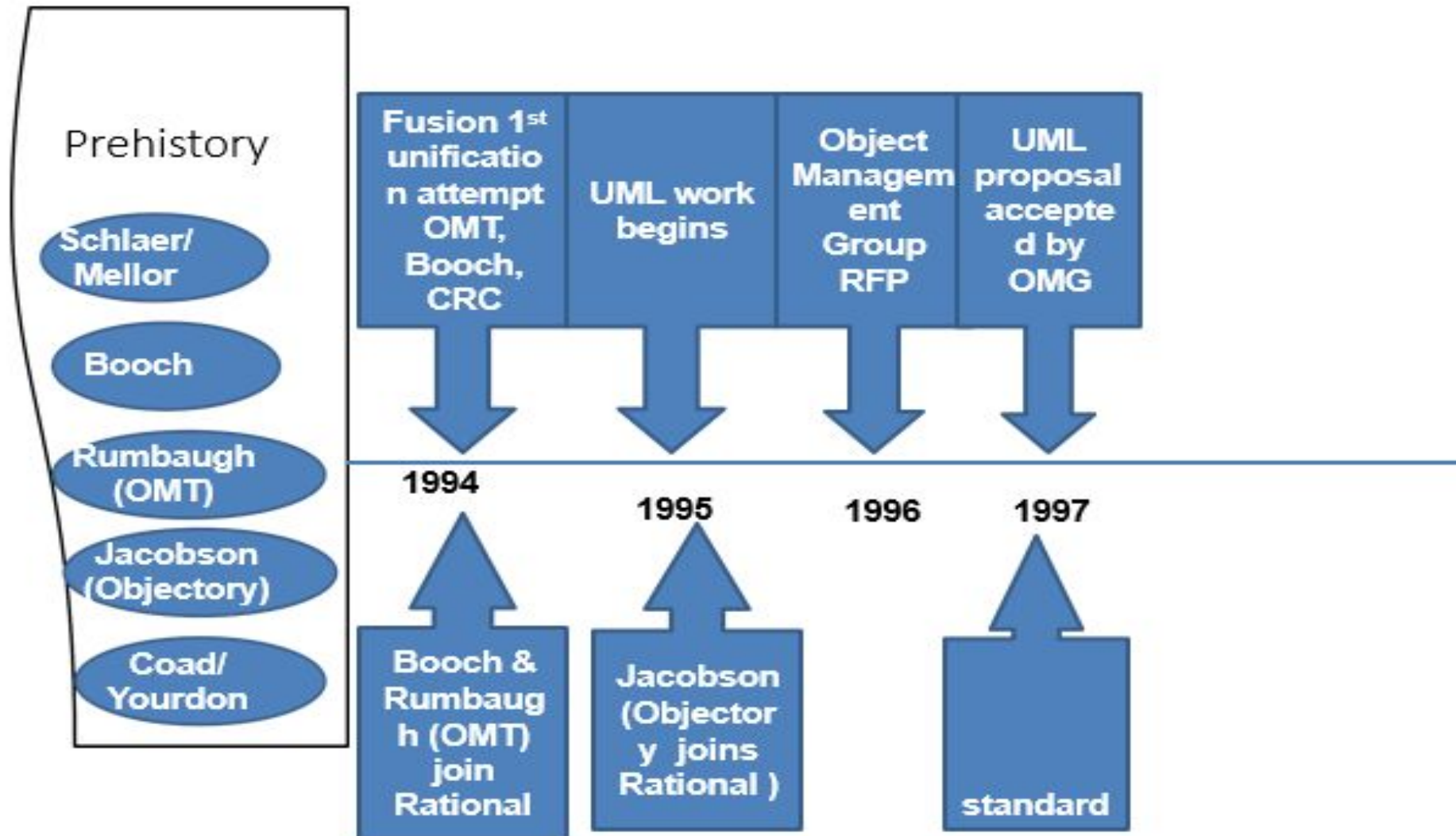
Need for Standardization → Enter UML.

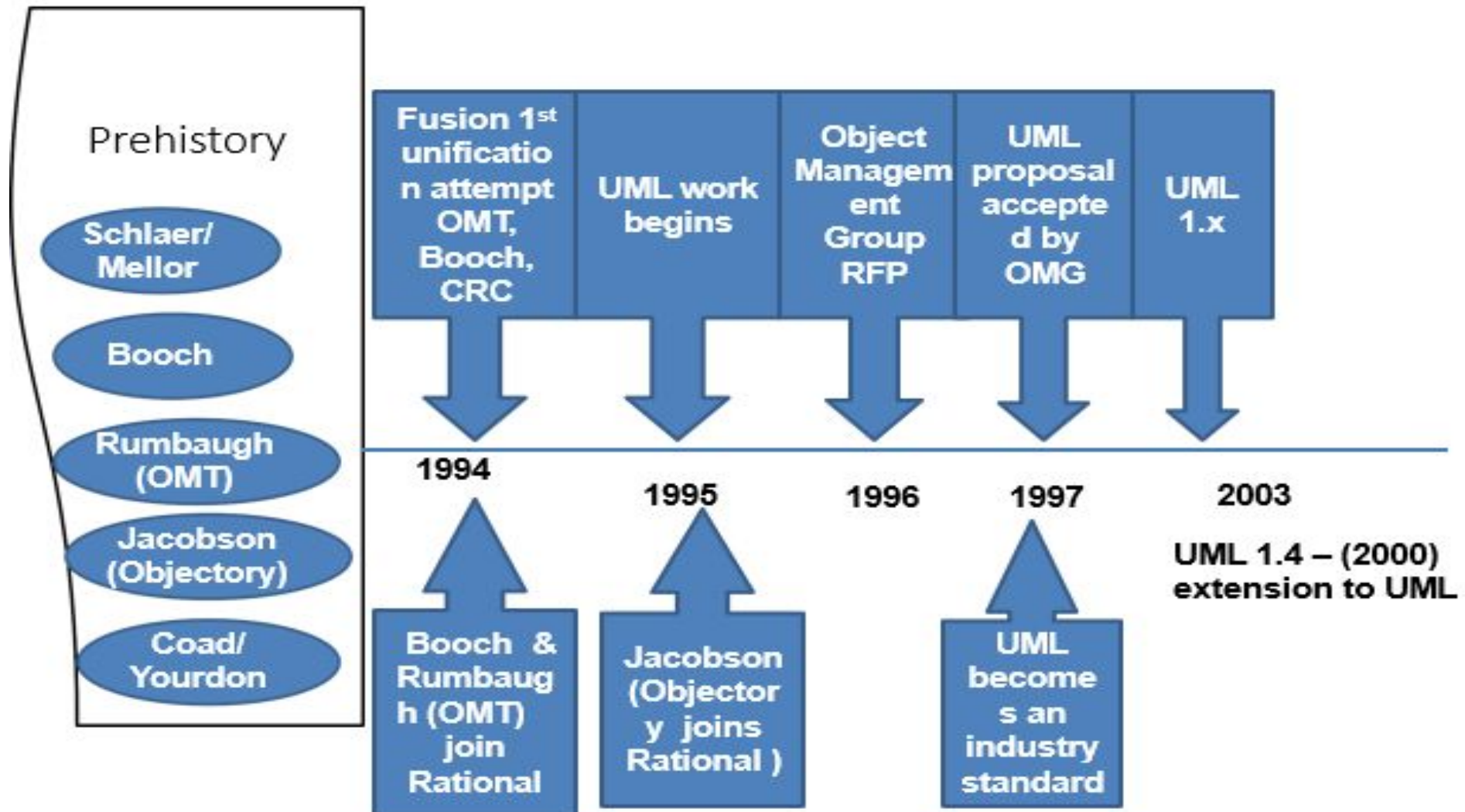
The Birth of UML

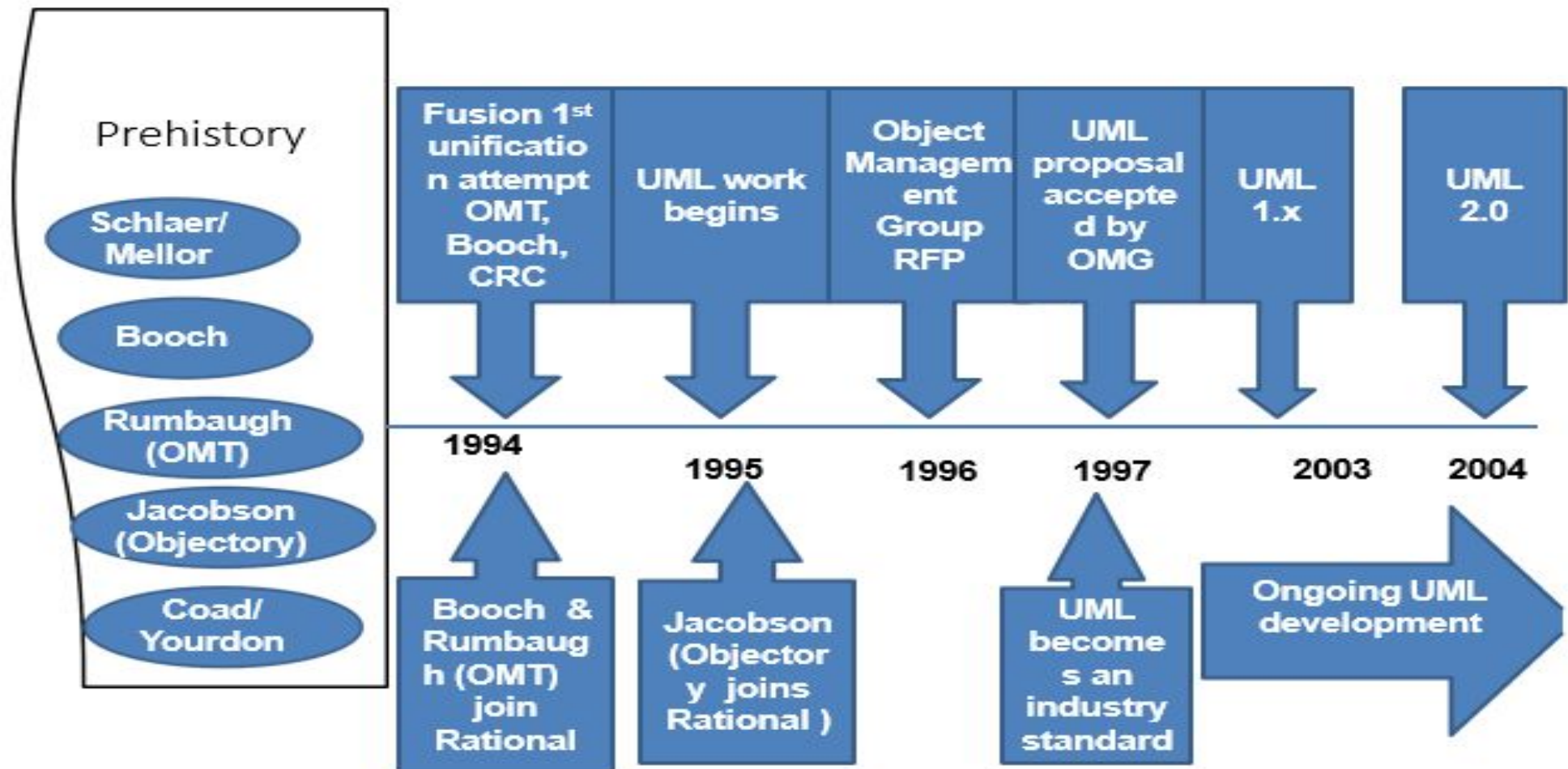












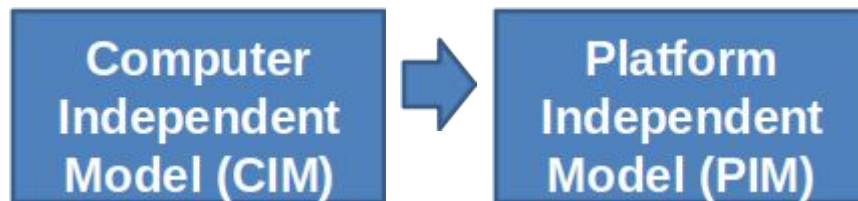
And today, UML is the industry standard for modeling software

The future of UML-MDA

- Future of UML defined by – MDA (Model Driven Architecture)
- MDA define a vision for how software can be developed based on models
- The production of the executable software architecture
- In MDA, software is produced through a series of model transformations aided by an MDA modeling tool

Computer Independent Model (CIM)

- CIM describes what the business wants.
- Very high level of abstraction
- Captures key requirements of the system and the vocabulary of the problem domain
- Computer Independent
- Creation is optional



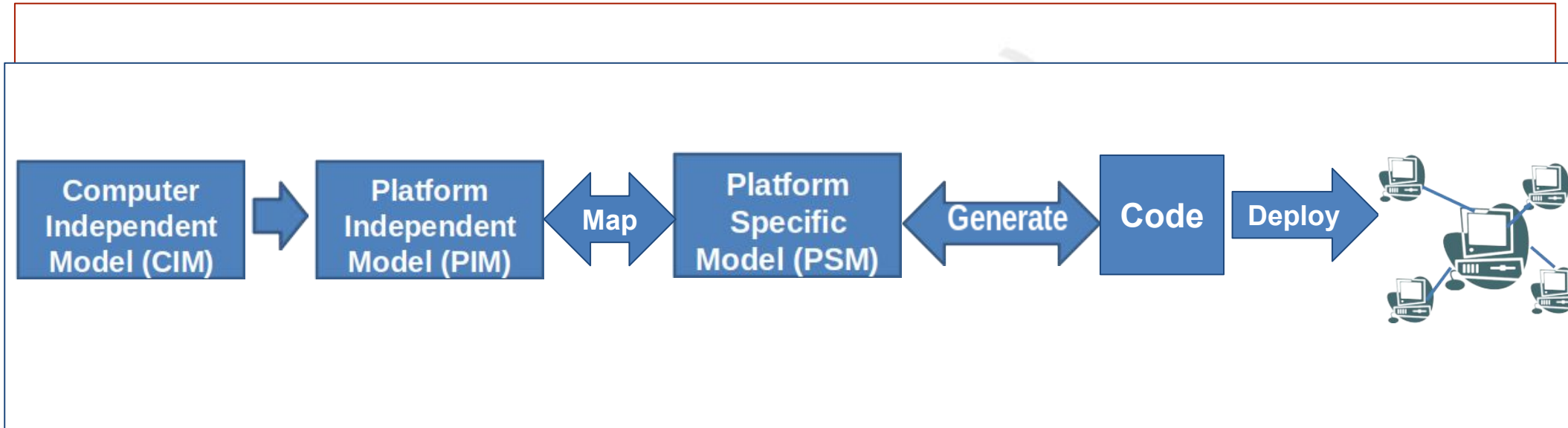
- Expresses the business semantics of the software system
- Independent of any underlying platform
- Same level of abstraction
- More complete
- Provide basis to transformed into PSM



- Map with PIM to create the PSM
- Source code is generated from the PSM

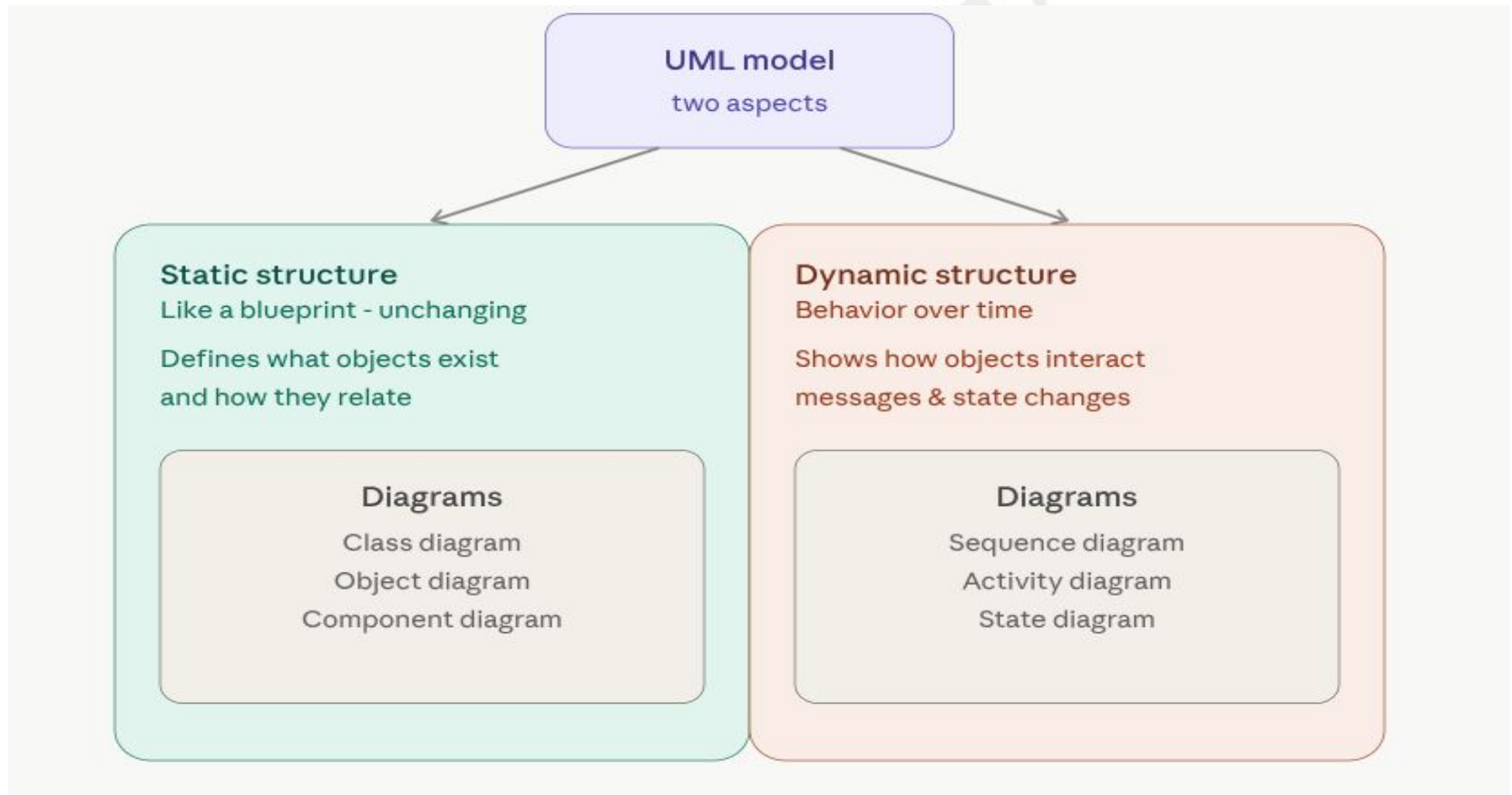


- Source code such as Java and C# code, is just the machine code resulting from the compilation of UML models
- This code generated as needed directly from PSM

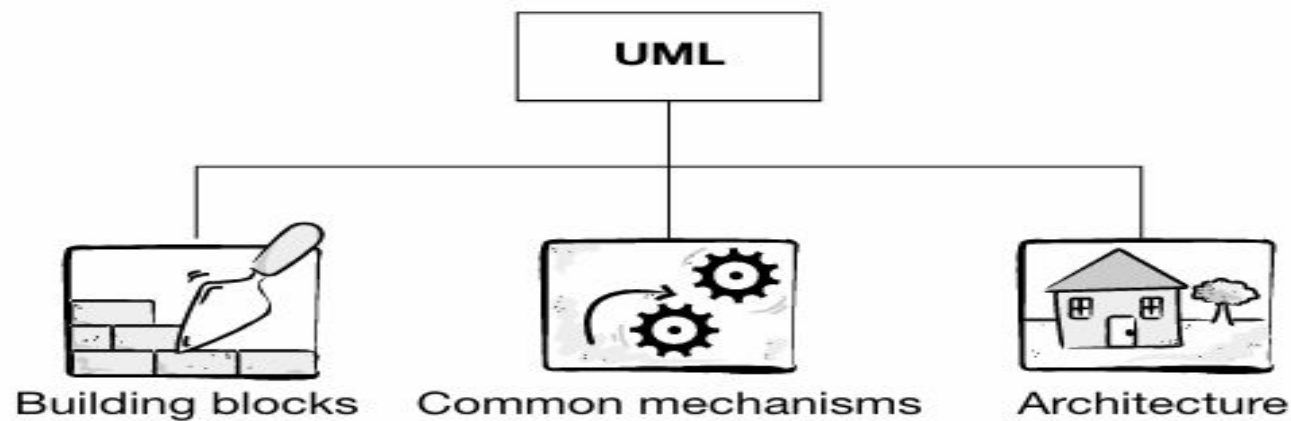


- MDA shifts UML models from their current role – manually created source code into the primary mechanism of code production

Objects and UML

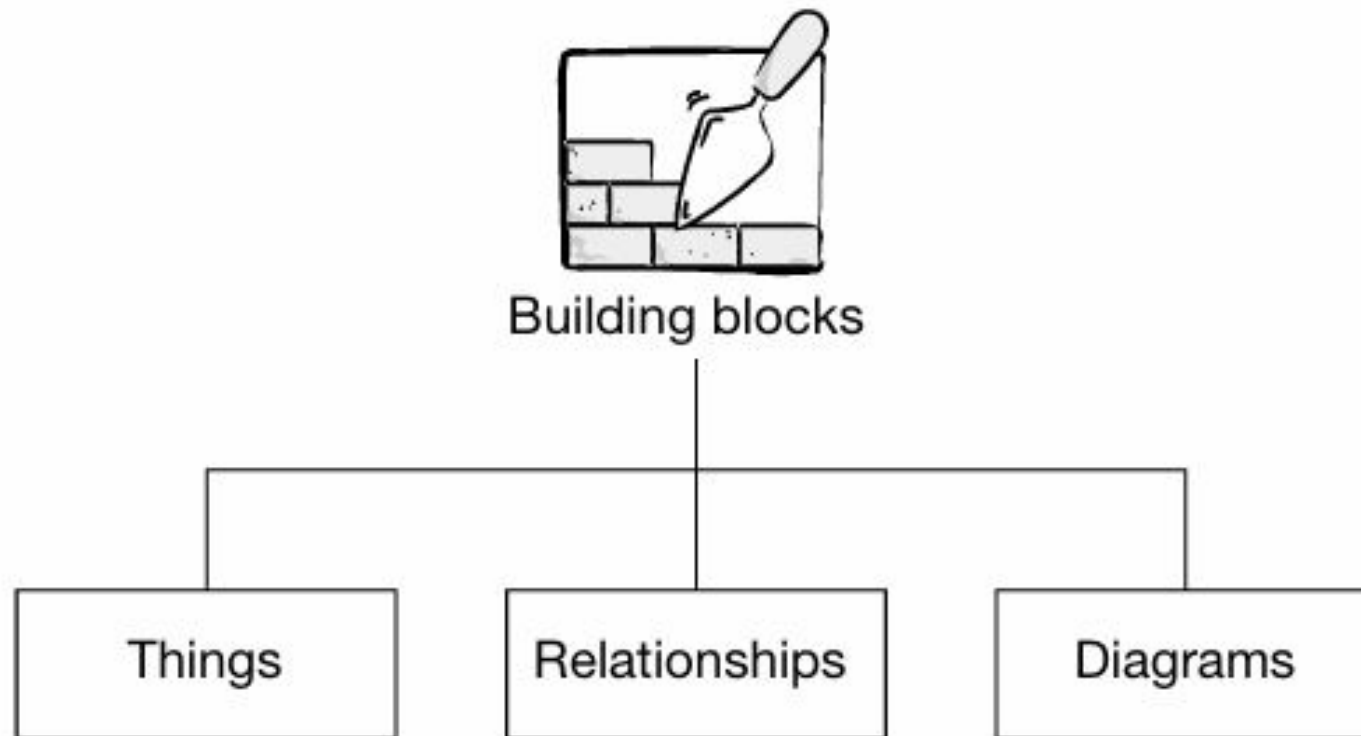


UML Structure



- building blocks – these are the basic UML modeling elements, relationships and diagrams
- common mechanisms – common UML ways of achieving specific goals;
- architecture – standard view of a system's architecture using different viewpoints

UML Building Blocks



Things (main entities)

- These are the modeling elements that are represented in UML diagrams.

structural

- The structural things define the static part of Model.
- Eg. Class, Use Case, Interface, Component

Behavioral

- The behavioral things consists of dynamic part of Model.
- Eg. Interaction, state machine

Grouping

- Grouping things are the organizational parts of UML Model.
- Eg. Packages

Annotational

- Annotational things are the explanatory part of UML Model.
- Eg. Note

Relationships

- How elements/things are associated with each other.
- It is used to show interaction within the elements in the system.

Types of Relationship –

1. Association
2. Aggregation
3. Composition
4. Generalization
5. Realization
6. Dependency

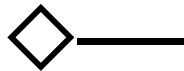
Association

- This relationship describes a set of link.
- It also describes how many objects are taking part in the relationship.
- No ownership
- Symbol: A plain solid line 
- Direction (optional): Arrow at one or both ends
- Example:
Customer  Product
(Customer buys Product)

Aggregation

- A whole-part relationship, but the part can exist independently of the whole. (weak "Has-a" relationship)

- Symbol: A hollow diamond at the whole side



- Weak ownership

Example:

ShoppingCart ◇— Product

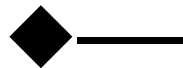
(The Shopping Cart has Products. But if the cart is deleted, Products still exist in Amazon)

Composition

- A stronger form of aggregation, where the part cannot exist without the whole.

Strong has-a relationship

- Symbol: A filled (black) diamond at the whole side



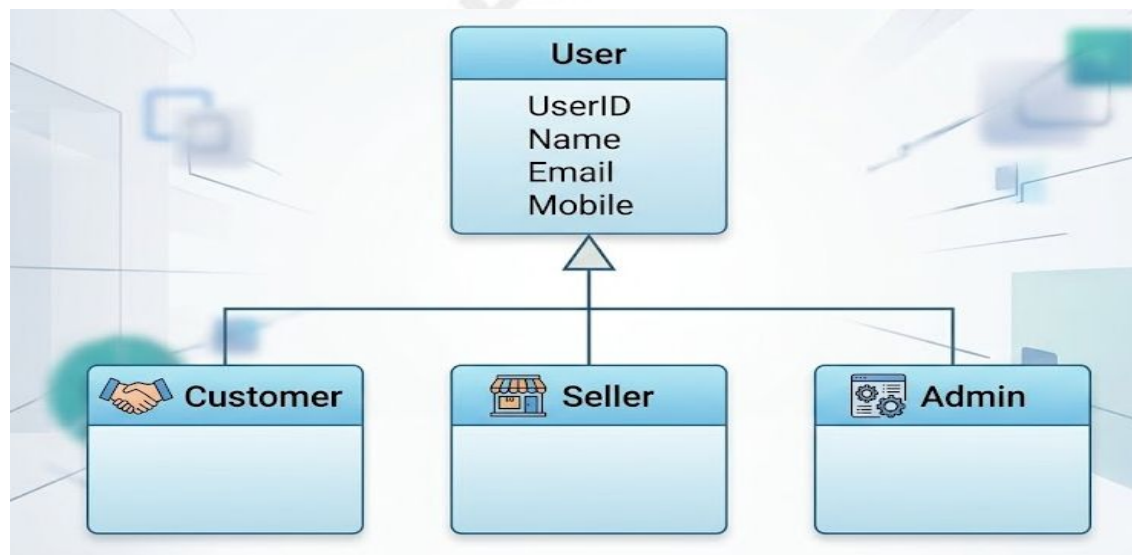
- Type: Strong ownership

- Example:

Order  OrderItem (The order items cannot exist without that order.)

Generalization

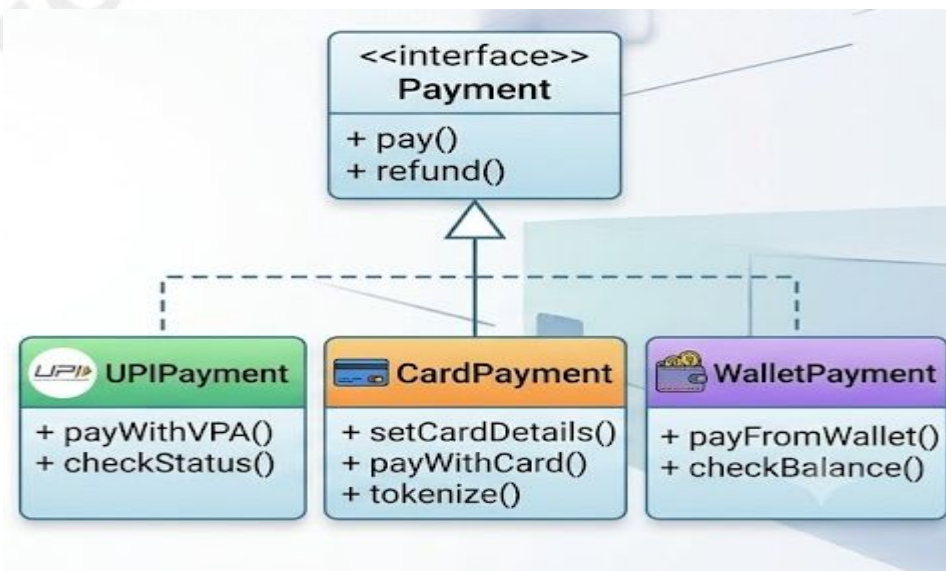
- Generalization is a relationship in which object of specialized elements are substitutable for object of the generalized element.
- Used to show inheritance concept.
- Symbol: A solid line with a hollow arrowhead pointing to the parent class.



Realization

- Realization is a semantic relationship which specifies contract that another element guarantees to carry out.
- Used when a class implements an interface.
- Interface = Rules/Contract
- Class = Person following those rules
- Symbol: A dashed line with a hollow triangle arrowhead pointing to the interface.

-----▷



Dependency

- Dependency is the relationship between two things in which change in one element also affects the another one.
- Symbol: A dashed line with a regular arrowhead pointing to the depended-upon element.

- - - - - ►

- Example:

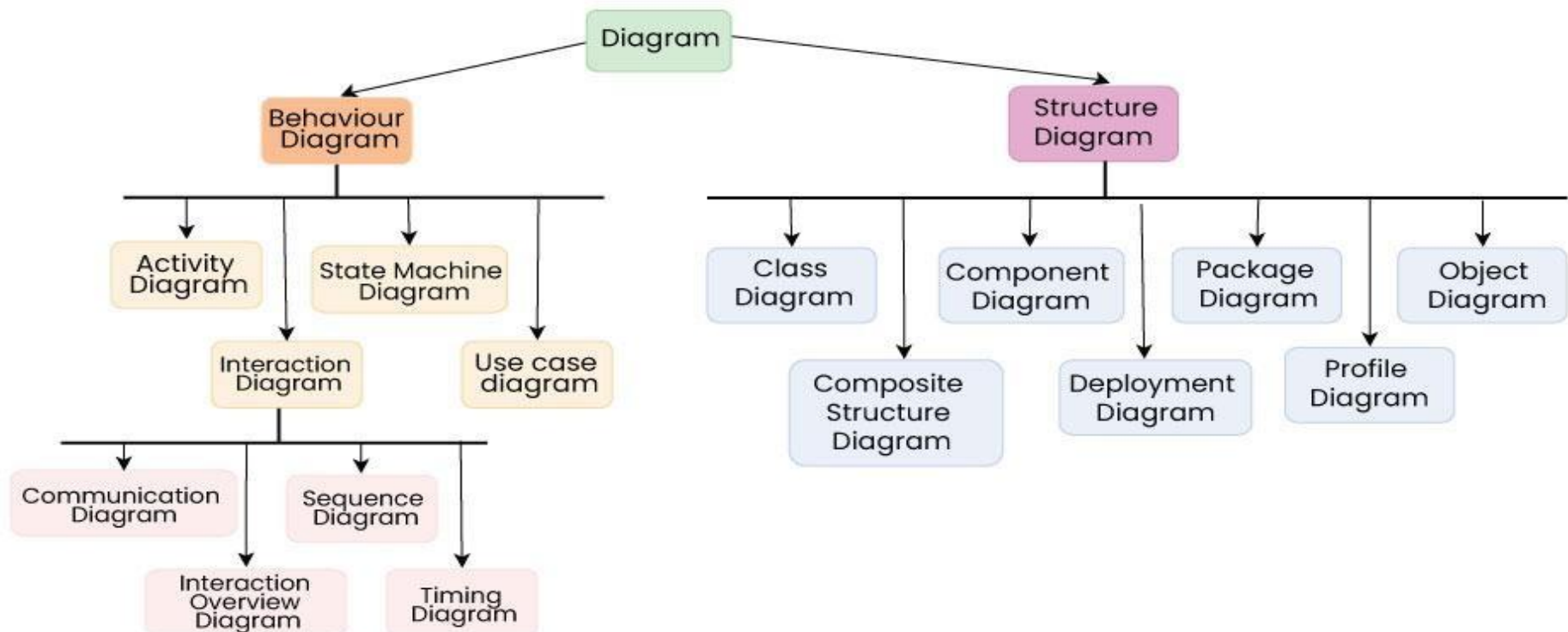
Invoice Service - - - - - ► PDF Generator

OTP Service - - - - - ► SMS Gaateway

Diagrams

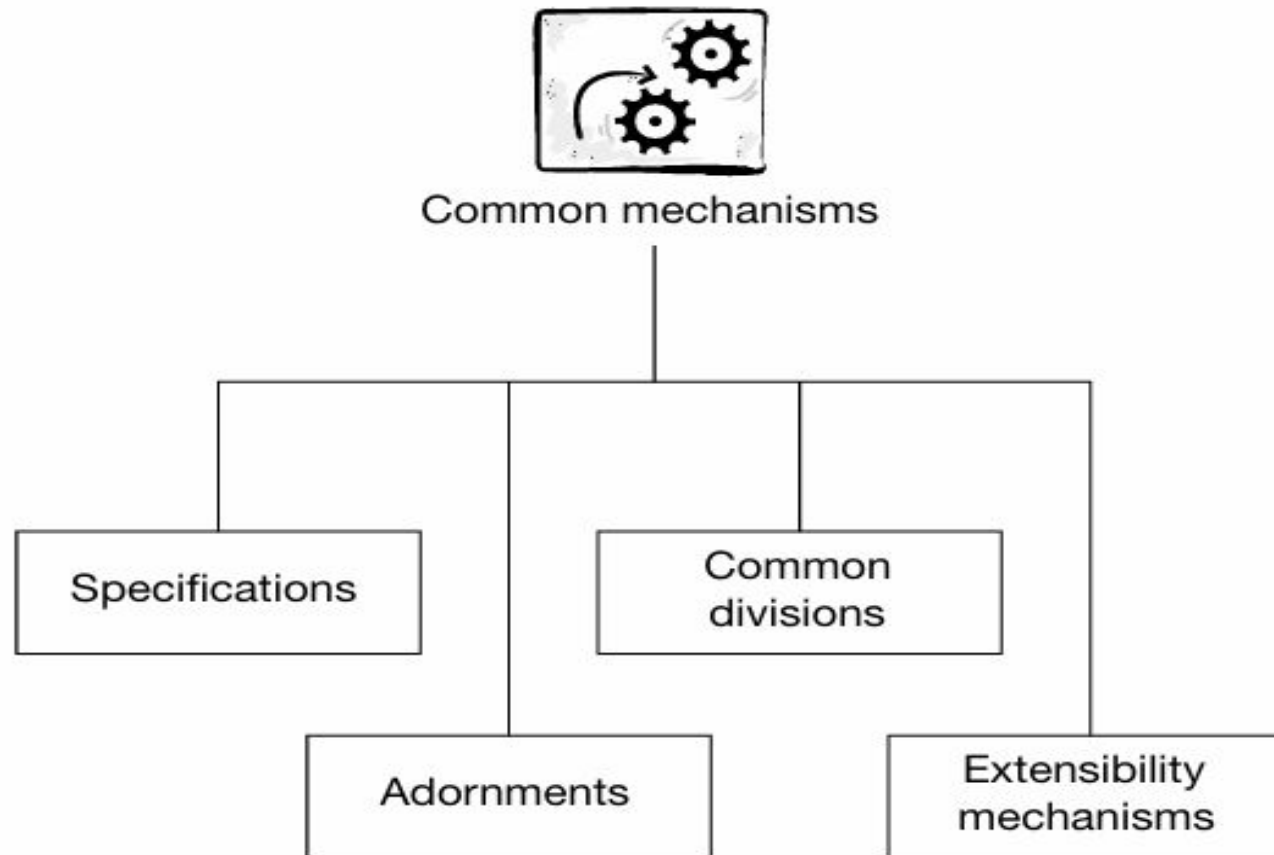
Structural Diagrams : capture static aspects or Structure of a System.

Behavioral Diagrams : Capture dynamic aspects or behavior of a System.



UML common mechanisms

Common Mechanisms are special features provided by UML to make diagrams more expressive, informative, and flexible.



Specifications

- Two dimensions

1. **Graphical** : allow you to model using diagrams and icons

2. **Textual** : consists of the specifications of the various modeling elements.

Specifications are textual descriptions of the semantics of an element.

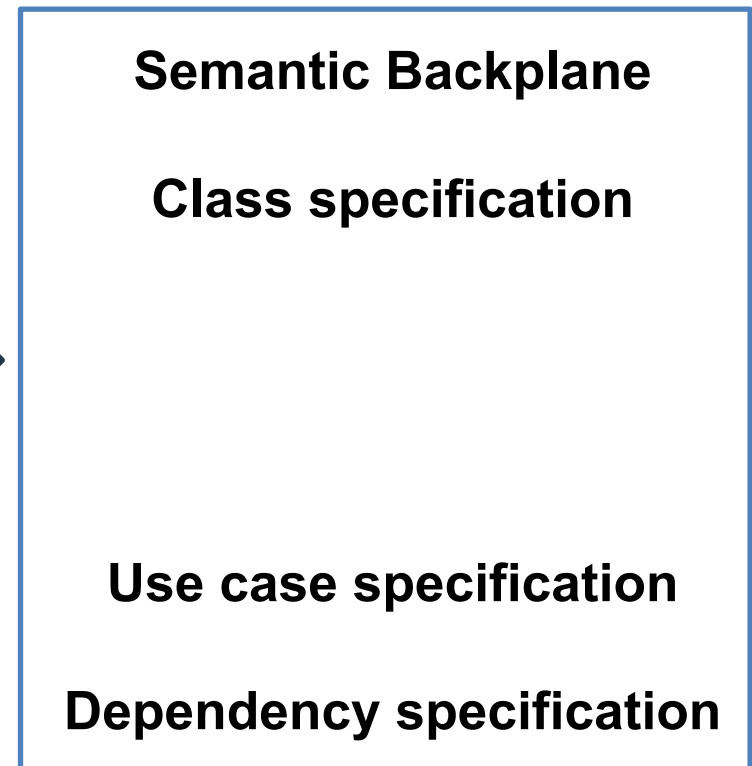
They provide the semantic plan of the model

The set of specification is the real “meat” of the model, and forms the semantic backplane that holds the model together and gives it meaning.

For Example :



Doesn't tell anything
about business
semantics



Model may be:

- Elided – some element are present in backplane, but hidden in diagram
- Incomplete – some missing elements of the model
- Inconsistent – model may contain contradictions

For a model to be considered in any way useful or complete the model semantics must be present in the backplane.

Adornment

- Adorn model elements on UML diagrams to highlight important details
- Constructing a high-level diagram
- Show only those adornments that highlight important features of the model and that increase the overall clarity and readability of the diagram

For Example -

Window

Unadorned element

Adornment

Meaning

+ / - / #

Public / Private / Protected

<<interface>>

Indicates an interface

{abstract}

Shows a class is abstract

{readOnly}

Attribute cannot be changed

Stereotypes

Tags like <<controller>>

Window
{author=Jim,
status=tested}

+size : Area(100,100)
 #visibility : Boolean =
 false
 +defaultSize : Rectangle
 #maximumSize :
 Rectangle

+create()
 +hide()
 +display(location : Point)

Element with adornments

Note

- Graphical symbol for rendering constraints or comments attached to an element or collection of elements.
- No Semantic Impact: does not alter the meaning of model.
- Specify things like: Requirements, Observations, Reviews, Explanations, Constraints.
- Graphically, a note is rendered as a rectangle with a dog-eared corner, together with a textual or graphical comment.



**SWA Online
Common Business
Components**

Owner: H. Jordan

Common Divisions

- Common Division refers to the practice of logically separating elements in UML into meaningful pairs or roles to make models clear, maintainable, and structured.

It helps to:

- Clarify roles and responsibilities
- Improve abstraction
- Support separation of concerns
- Promote flexibility and maintainability

- Two common divisions in UML – classifier and instance, and interface and implementation

1. Classifier and Instance

Classifier – an abstraction of grouping similar object (a type or blueprint)

e.g. bank account

Instance – concrete things or instance of that abstraction

e.g. your bank account, my bank account

2. Interface and Implementation

Interface – define rules for interaction

- What something should do
- e.g. the button on the front of your VCR

Implementation – realization of the rules define in the interface

- how it does
- e.g. the mechanism inside VCR

Classifier	Semantics
Actor	Role played by an outside user of the system
Class	Description of a set of objects
Component	Modular or replaceable part of a system
Interface	Collection of operations
Node	Physical, runtime element
Signal	Message passed between objects
Use case	Sequence of actions

Extensibility mechanisms

- UML allows extending the language to suit specific needs — via stereotypes, tagged values, and constraints.
- 1. **Constraints** - Constraints allow you to add new rules to modeling elements
 - Use text string in braces { }
- 2. **Stereotypes**- Stereotypes allow you to define new modeling elements
 - Write name in guillemets <<...>>
- 3. **Tagged values** - Tagged values allow you to add your own properties to model elements
 - Write like { tag1=value1, tag2=value2, ... ,tagN=valueN}

Stereotypes

- Stereotypes are used to create new building blocks from existing blocks
- New building blocks are domain-specific
- Stereotypes are used to extend the vocabulary of a system
- Graphically represented as a name enclosed by guillemets (« »)
- Example:
<<controller>>, <<entity>>, <<boundary>>, <<interface>>, <<service>>, <<datastore>>

Use:

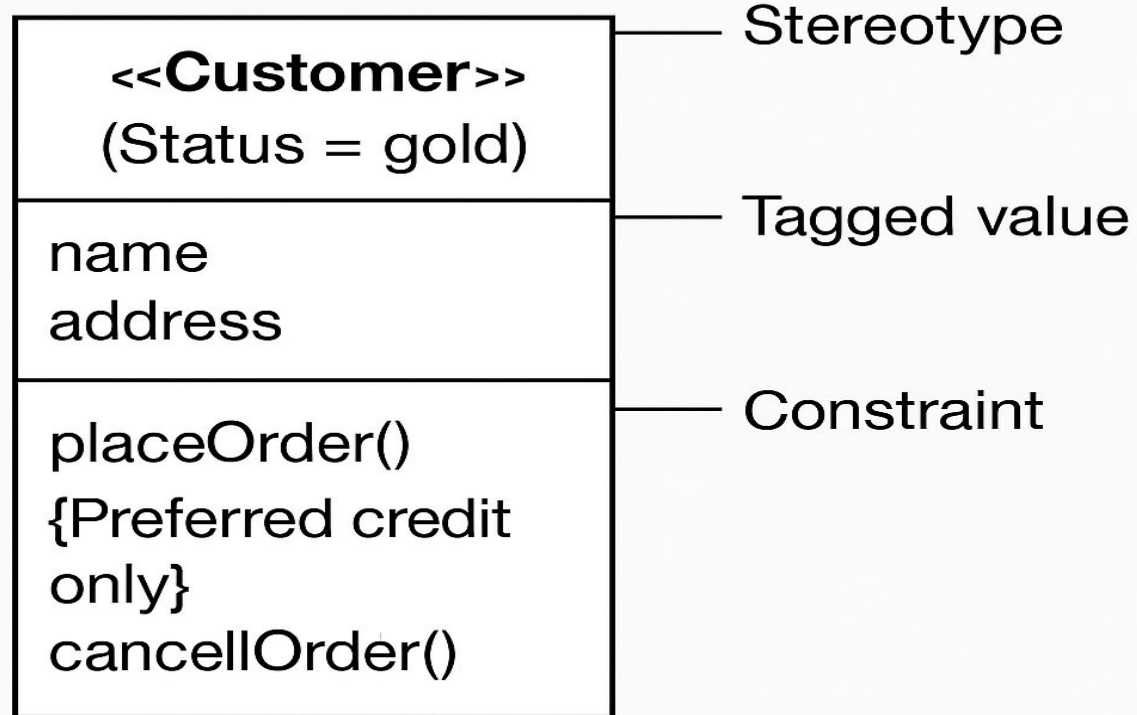
- Clarify role or responsibility
- Extend UML element with domain-specific semantics

Tagged Values

- Tagged values are used to add to the information of the element (not of its instances)
- Stereotypes help to create new building blocks, whereas tagged values help to create new attributes
- These are commonly used to specify information relevant to code generation, configuration management and so on

Constraints

- Constraints are used to create rules for the model
- Rules that impact the behavior of the model, and specify conditions that must be met
- Can apply to any element in the model, i.e., attributes of a class, relationship
- Graphically represented as a string enclosed by braces {...} and placed near the associated elements or connected to that elements by dependency relationships



UML Architecture

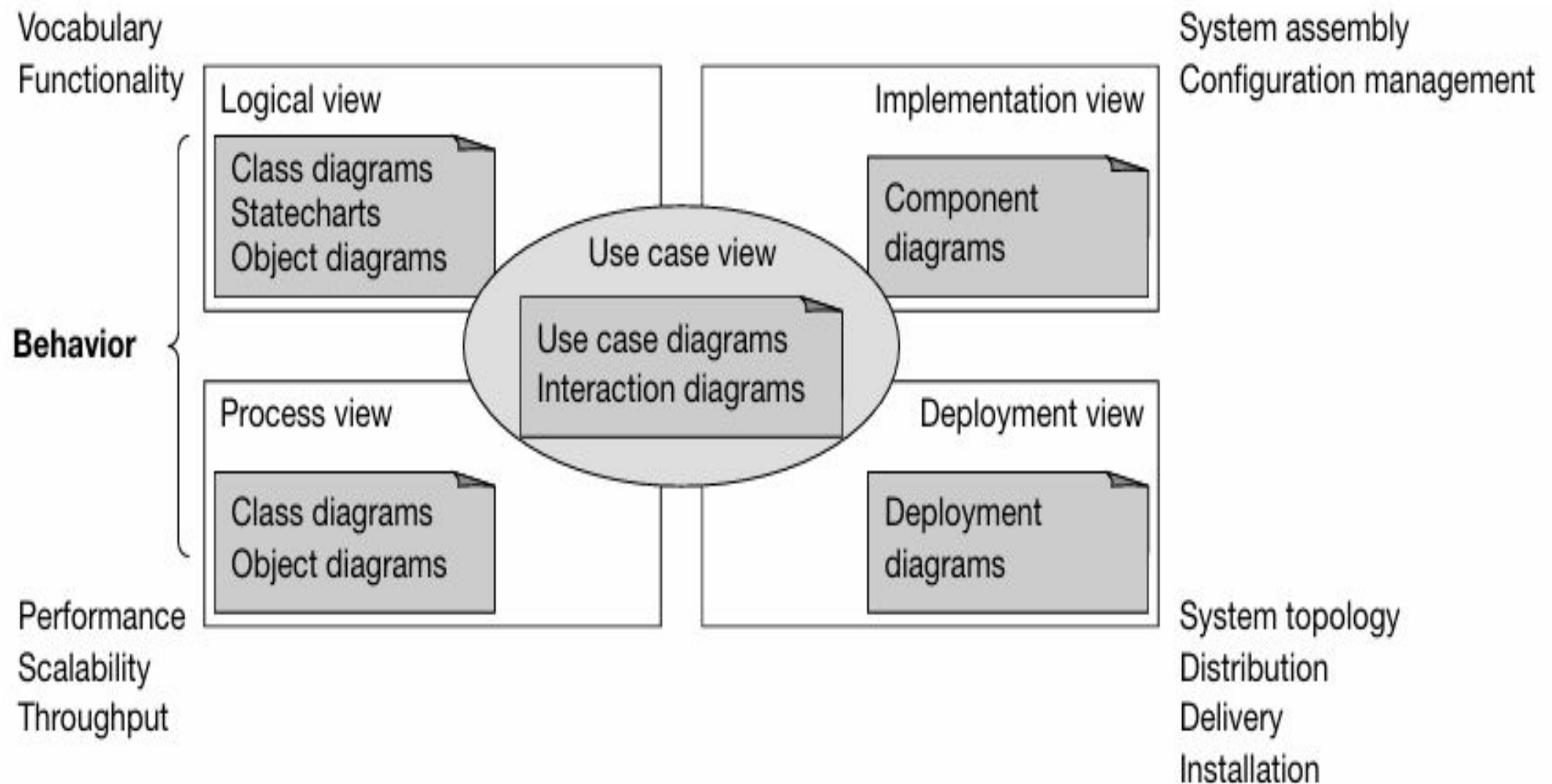
- The UML (Unified Modeling Language) architecture defines how the language itself is structured, not just how it's used to model systems.
- Any real-world system is used by different users. The users can be developers, testers, business people, analysts, and many more. Hence, before designing a system, the architecture is made with different perspectives in mind. The most important part is to visualize the system from the perspective of different viewers. The better we understand the better we can build the system.

UML Architecture

- **“ 4+1 View” architecture also known as the Kruchten’s 4+1 View Model, described by philippe Kruchten**

4 views = Logical, Development, Process, Physical

+1 view = Use Case view (central to all others)



Logical View

- Shows functionality of the system from an object-oriented perspective
- Describes how the system is broken into classes and objects
- Provide functionality to end user

UML Diagrams Used: Class Diagram, Object Diagram, Communication Diagram

Process View

- Describes dynamic aspects of the system
- Models the executable threads and processes in a system as active classes
- Concerns with performance, scalability and throughput

UML Diagrams Used: Activity Diagram, Sequence Diagram, State Machine Diagram

Implementation View

- Describes the software architecture in terms of modules/components
- Focuses on code organization
- It shows how the system is built in terms of code and files.

UML Diagrams Used: Component Diagram , Package Diagram

Deployment View

- Describes how the system is deployed physically
- Concerns with system topology, distribution, delivery and installation

UML Diagrams Used: Deployment Diagram

Use case View

- Capture the basic requirements for the system as a set of use case
- Defines scenarios of system behavior
- Guides all other views

UML Diagrams Used: Use Case Diagram, Activity Diagram, Sequence Diagram

MCQ's

1. UML is primarily used for:

- a) Coding
- b) Testing
- c) Modeling software systems
- d) Designing UI

Answer: c

2. Who maintains UML standards?

- a) ISO
- b) Oracle
- c) IEEE
- d) OMG (Object Management Group)

Answer: d

3. Which of the following is a structural diagram?

- a) Activity Diagram
- b) Use Case Diagram
- c) Class Diagram
- d) Sequence Diagram

Answer: c

MCQ's

4. Which of the following is not a category of UML “Things”?

- a) Structural
- b) Behavioral
- c) Functional
- d) Annotational

Answer: c

5. Which mechanism provides visual decorations like <<interface>> or {abstract}?

- a) Extensibility
- b) Adornments
- c) Grouping
- d) Specification

Answer: b

MCQ's

6. Which of the following is not a type of UML relationship?

- a) Association
- b) Aggregation
- c) Inheritance
- d) Realization

Answer: c

7. Which view in UML architecture addresses the structure and design of the software?

- a) Logical view
- b) Process view
- c) Development view
- d) Physical view

Answer: a

MCQ's

8. What does the '+1' in the 4+1 View Model represent?

- a) User Interface
- b) Deployment
- c) Scenarios or Use Cases
- d) Class Diagrams

Answer: c

9. In UML, the Physical view is concerned with:

Class organization

Component packaging

Network and deployment configuration

Thread management

- a)
- b)
- c)
- d)

Answer: c

True/False Questions

1. UML diagrams are part of UML building blocks.

True

2. Aggregation is a stronger relationship than composition in UML.

False

3. Generalization represents an "is-a" relationship.

True

4. Realization is used to show interface implementation in UML.

True

THANK YOU